

cross_provider_sharing

Cross-Provider Model Sharing Guide

Overview

The HelixCode LLM system provides comprehensive cross-provider model sharing functionality that enables:

- **Universal Model Access:** Download a model once and use it across all compatible providers
- **Automatic Format Conversion:** Convert models between formats (GGUF, GPTQ, AWQ, HF, etc.)
- **Provider Optimization:** Optimize models specifically for target providers
- **Intelligent Compatibility:** Automatic detection of provider-model compatibility
- **Seamless Integration:** All providers work together without manual configuration

Supported Providers

Provider	Type	Formats	Use Case	Optimization Target
VLLM	OpenAI-compatible	GGUF, GPTQ, AWQ, HF, FP16, BF16	High-throughput GPU inference	
Llama.cpp	Custom	GGUF	CPU/GPU universal inference	
Ollama	OpenAI-compatible	GGUF	User-friendly model management	
LocalAI	OpenAI-compatible	GGUF, GPTQ, AWQ, HF	Drop-in OpenAI replacement	
FastChat	OpenAI-compatible	GGUF, GPTQ, HF		

Provider	Type	Formats	Use Case	Optimization Target
			Training and serving platform	
TextGen	OpenAI-compatible	GGUF, GPTQ, HF	Popular Gradio interface	
LM Studio	OpenAI-compatible	GGUF, GPTQ, HF	User-friendly desktop app	
Jan AI	OpenAI-compatible	GGUF, GPTQ, HF	Open-source AI assistant	
KoboldAI	Custom API	GGUF	Writing-focused interface	
GPT4All	OpenAI-compatible	GGUF	CPU-focused inference	
TabbyAPI	OpenAI-compatible	GGUF, GPTQ, HF	High-performance server	
MLX	OpenAI-compatible	GGUF, HF	Apple Silicon optimization	
MistralRS	OpenAI-compatible	GGUF, GPTQ, HF, BF16, FP16	Rust-based high performance	

Key Features

1. Universal Model Registry

Centralized registry tracks all models and their compatibility:

- **Model Metadata:** Size, context, capabilities, requirements
- **Format Support:** Which formats each provider supports
- **Conversion Paths:** Available conversion methods between formats
- **Performance Characteristics:** Throughput, latency, memory usage
- **Hardware Requirements:** CPU/GPU needs, RAM/VRAM requirements

2. Intelligent Compatibility Checking

Automatically determines if a model can work with a provider:

```
# Check compatibility before downloading
helix local-llm models download llama-3-8b-instruct --provider
vllm
```

3. Cross-Provider Model Sharing

Share models across all compatible providers:

```
# Share a downloaded model with all providers
helix local-llm share ./models/llama-3-8b.gguf

# Share with specific provider only
helix local-llm share ./models/llama-3-8b.gguf --provider vllm
```

4. Universal Model Download

Download once, use everywhere:

```
# Download and share with all compatible providers
helix local-llm download-all llama-3-8b-instruct

# Download in specific format
helix local-llm download-all mistral-7b-instruct --format gguf
```

5. Provider-Specific Optimization

Optimize models for maximum performance:

```
# Optimize for VLLM (GPU performance)
helix local-llm optimize ./model.gguf --provider vllm

# Optimize for Llama.cpp (CPU compatibility)
helix local-llm optimize ./model.hf --provider llamacpp

# Optimize for MLX (Apple Silicon)
helix local-llm optimize ./model.gguf --provider mlx
```

6. Full Synchronization

Synchronize all models across all providers:

```
# Sync all downloaded models
helix local-llm sync

# This will:
# - Scan all provider directories
# - Share compatible models
# - Convert incompatible models when possible
# - Report any issues
```

Format Support Matrix

From To	GGUF	GPTQ	AWQ	HF	FP16	BF16
HF	☑	⚠	⚠	✗	✗	✗
FP16	✗	☑	⚠	✗	✗	✗
BF16	✗	⚠	☑	✗	✗	✗
GGUF	☑	⚠				
GPTQ						
AWQ						

Legend: - ☑ Direct support - ⚠ Conversion required (quality loss possible) - ✗ Not supported

Conversion Tools

GGUF Conversion

- **Tool:** llama.cpp
- **Source Formats:** HF, FP16, BF16
- **Optimization:** CPU/GPU universal
- **Use Case:** Maximum compatibility

GPTQ Conversion

- **Tool:** AutoGPTQ
- **Source Formats:** HF, FP16, BF16
- **Optimization:** NVIDIA GPU
- **Use Case:** High-performance GPU inference

AWQ Conversion

- **Tool:** AutoAWQ

- **Source Formats:** HF, FP16, BF16
- **Optimization:** GPU
- **Use Case:** Balanced performance/quality

Hardware Optimization

CPU Optimization

Providers: Llama.cpp, Ollama, GPT4All

- **Format:** GGUF
- **Quantization:** Q4_K_M, Q5_K_M
- **Features:** CPU-specific optimizations, low memory usage

GPU Optimization

Providers: VLLM, FastChat, TabbyAPI

- **Format:** GPTQ, AWQ, GGUF
- **Quantization:** Q4, Q8
- **Features:** CUDA/ROCm support, batch processing

Apple Silicon Optimization

Providers: MLX, VLLM (with Metal)

- **Format:** GGUF, FP16
- **Quantization:** Q4_K_M, Q8_0
- **Features:** Metal acceleration, unified memory

Usage Examples

Basic Workflow

```
# 1. Initialize all providers
helix local-llm init

# 2. Start providers
helix local-llm start

# 3. Download a model for all providers
helix local-llm download-all llama-3-8b-instruct

# 4. Check status
```

```
helix local-llm status
```

```
# 5. List shared models
```

```
helix local-llm list-shared
```

Advanced Workflow

```
# 1. Download specific model
```

```
helix local-llm models download codellama-7b-instruct --format hf
```

```
# 2. Convert to optimal format
```

```
helix local-llm models convert ./codellama-7b.hf --format gguf --  
    quantize q4_k_m
```

```
# 3. Share across providers
```

```
helix local-llm share ./codellama-7b.gguf
```

```
# 4. Optimize for high-performance provider
```

```
helix local-llm optimize ./codellama-7b.gguf --provider vllm
```

```
# 5. Sync everything
```

```
helix local-llm sync
```

Provider-Specific Usage

```
# VLLM (High-throughput GPU)
```

```
helix local-llm start vllm
```

```
helix local-llm optimize ./model.gguf --provider vllm
```

```
# Llama.cpp (CPU/GPU universal)
```

```
helix local-llm start llamacpp
```

```
helix local-llm share ./model.gguf --provider llamacpp
```

```
# Ollama (User-friendly)
```

```
helix local-llm start ollama
```

```
helix local-llm download-all mistral-7b-instruct --format gguf
```

```
# MLX (Apple Silicon)
```

```
helix local-llm start mlx
```

```
helix local-llm optimize ./model.hf --provider mlx
```

Troubleshooting

Common Issues

1. Model Not Compatible

Error: format gptq is not compatible with provider llamacpp

Solution: Convert to compatible format:

```
helix local-llm optimize ./model.gptq --provider llamacpp
```

2. Conversion Failed

Error: conversion failed: insufficient memory

Solution: Use lighter quantization:

```
helix local-llm convert ./model.hf --format gguf --quantize q4_0
```

3. Provider Not Starting

Error: provider failed to start: port already in use

Solution: Check ports and stop conflicting providers:

```
helix local-llm stop  
helix local-llm status
```

Performance Optimization

1. Reduce Memory Usage

```
# Use more aggressive quantization  
helix local-llm convert ./model.hf --format gguf --quantize  
    q3_k_m
```

2. Increase Speed

```
# Optimize for specific provider  
helix local-llm optimize ./model.gguf --provider vllm
```

3. Improve Quality

```
# Use higher quality quantization  
helix local-llm convert ./model.hf --format gptq --quantize q8_0
```

Best Practices

1. Model Management

- **Use GGUF for maximum compatibility** across providers
- **Download once, share everywhere** to save bandwidth
- **Regular sync** to ensure all providers have latest models
- **Provider-specific optimization** for best performance

2. Hardware Utilization

- **Match provider to hardware:** VLLM for NVIDIA, MLX for Apple, Llama.cpp for CPU
- **Optimize quantization** based on available memory
- **Use GPU providers** for high-throughput scenarios
- **Use CPU providers** for broad compatibility

3. Workflow Optimization

- **Initialize once:** `helix local-llm init` sets up everything
- **Batch operations:** Use `download-all` for multiple models
- **Regular maintenance:** Use `sync` to keep everything updated
- **Monitor status:** Use `status` to track provider health

API Integration

Programmatic Usage

```
package main

import (
    "context"
    "fmt"
    "dev.helix.code/internal/llm"
)

func main() {
    ctx := context.Background()

    // Initialize manager
    manager := llm.NewLocalLLMManager("/path/to/local-llm")
    manager.Initialize(ctx)

    // Download and share model
```



```

err := manager.DownloadModelForAllProviders(ctx, "llama-3-8b-
instruct", llm.FormatGGUF)
if err != nil {
    panic(err)
}

// Get shared models
shared, err := manager.GetSharedModels(ctx)
if err != nil {
    panic(err)
}

fmt.Printf("Shared models: %+v\n", shared)
}

```

Integration with HelixCode

The cross-provider system integrates seamlessly with HelixCode:

- **Automatic provider selection** based on task requirements
- **Dynamic model loading** from optimal provider
- **Fallback mechanisms** if providers are unavailable
- **Performance monitoring** and automatic optimization

Future Enhancements

Planned Features

- **Distributed model sharing** across multiple machines
- **Automatic model updates** from sources
- **Performance benchmarking** across providers
- **Cost optimization** for cloud deployments
- **Advanced caching** strategies
- **Model versioning** and rollback

Community Contributions

- **New provider integrations** welcomed
- **Conversion tool improvements** encouraged
- **Performance optimizations** appreciated
- **Documentation enhancements** valued

Conclusion

The HelixCode cross-provider model sharing system provides:

- **Universal Access:** Any model, any provider
- **Automatic Optimization:** Best performance without manual intervention
- **Intelligent Management:** Smart compatibility and conversion
- **Seamless Integration:** Works with existing HelixCode workflows
- **Future-Proof Design:** Extensible architecture for new providers

This eliminates the traditional silos between LLM providers and enables a truly unified model ecosystem.